

Módulo 5: Herramientas low-code y no-code con IA (42 h)

Relación de fichas:

5.1. Introducción y Contexto Actual (6 h)

5.1.1 Low-code/no-code con IA: panorama, patrones y casos de uso (qué es y qué no es)

Marco general: qué automatiza un flujo low-code/no-code en empresa, patrones de activación (evento/programación/polling), casos típicos y papel real de la IA (clasificar, extraer, resumir, etiquetar) con límites operativos.

5.1.2 Datos en automatización: JSON estándar, campos, tipos, listas y validación básica

Cómo representar y “hacer viajar” los datos con un **formato estándar**: campos mínimos, tipos, arrays, metadatos; validación de entrada/salida y normalización (texto/fechas/nulos).

5.1.3 Diseño del proceso operable: evento vs programación, rutas (principal/revisión/error) y trazabilidad/KPIs

Cómo diseñar un flujo completo **implementable**: decisión evento vs batch, mapa de rutas, disparadores claros de revisión/error técnico, KPIs operativos y logging mínimo para medir y depurar.

5.1.4 Diseño de procesos automatizables: selección, requisitos y mapa AS-IS → TO-BE

Criterios para elegir un caso troncal viable (valor/riesgo/datos/complexidad), definición de alcance (incluye/excluye) y traducción a un mapa de proceso.

5.2. Estrategia y Comparativa: Make vs n8n (6 h)

5.2.1 Marco de decisión: cuándo elegir Make o n8n (criterios, pesos y trade-offs)

Metodología para decidir herramienta con criterios técnicos y operativos (flujo, datos/listas, HTTP, observabilidad, gobierno/entornos, volumen/coste) y cómo justificar la elección.

5.2.2 Modelo mental y terminología comparada (Make/n8n): equivalencias funcionales

Diccionario práctico Make ↔ n8n (scenario/workflow, module/node, bundle/item, router/IF, handlers/Error Trigger, etc.) para traducir el mismo flujo sin cambiar su lógica.

5.2.3 Comparativa estructurada + taller de defensa por casos (argumentario)

Tabla comparativa con implicaciones reales y cómo defender una decisión (criterio → implicación → decisión → riesgos → mitigación) para casos internos/externos y el caso troncal.

5.3. Make: construcción de automatizaciones (12 h)

5.3.1 [Make] Fundamentos y montaje del escenario: escenarios, módulos y disparadores (Scheduler/Webhook/Watch)

Qué es Make, cómo se estructura un escenario, cómo elegir y configurar triggers, cómo asegurar una primera salida verificable y reproducible.

5.3.2 [Make] Datos y lógica del flujo: mapping/funciones + Router/Filters + listas (bundles, Iterator/Aggregator)

Cómo normalizar datos al estándar, construir rutas de negocio y revisión, y tratar listas con eficiencia (sin disparar operaciones innecesarias).

5.3.3 [Make] Integraciones y operación mínima: HTTP, autenticación, errores, logging, alertas y lectura de ejecuciones

Cómo integrar sin conector con HTTP, manejar errores (401/403/429/5xx), diseñar rutas de error, registrar contexto mínimo y depurar con historial de ejecuciones.

5.4. n8n: construcción de automatizaciones (12 h)

5.4.1 [n8n] Fundamentos y montaje del workflow: workflows, trigger nodes (Webhook/Cron) y nodos de acción

Qué es n8n, estructura de workflow, activación/ejecuciones, triggers típicos y creación de una salida verificable con credenciales bien gestionadas.

5.4.2 [n8n] Datos y control de flujo: items + JSON + expressions + IF/Switch/Merge + Split In Batches

Normalización con Set/Expressions, diseño de rutas por prioridad/revisión, recombinación si procede y procesamiento de listas por lotes con trazabilidad.

5.4.3 [n8n] Integraciones y operación mínima: HTTP Request, errores, Error Trigger, executions y modularidad básica

Integración universal con HTTP Request, captura de errores, centralización con Error Trigger, depuración por executions y modularidad con sub-workflows cuando aporte mantenibilidad.

5.5. IA y operación mínima en producción (Make y n8n) (6 h)

5.5.1 IA en automatizaciones: patrones + contrato de entrada/salida (conector vs HTTP)

Cómo insertar un paso de IA “operable” (clasificación/extracción/resumen), definiendo exactamente qué se envía y qué debe volver (campos obligatorios y valores permitidos).

5.5.2 Calidad de salida IA: prompts operativos, validación, postprocesado y fallback (reintento o revisión)

Cómo redactar prompts para automatización (formato estricto), validar salidas IA, postprocesar al esquema estándar y activar fallback cuando la salida no cumple.

5.5.3 Checklist mínimo de producción + demo final + paquete de entrega (Make/n8n)

Checklist de credenciales, validación de entrada, logs/alertas, control de cambios/export, y demo obligatoria (1 OK + 1 fallo tratado) con evidencias y entrega reproducible.

5.1.1 Low-code/no-code con IA: panorama, patrones y casos de uso (qué es y qué no es)

Referencia

- **Módulo:** 5. Herramientas low-code y no-code con IA (42 h)
- **Apartado:** 5.1. Introducción y Contexto Actual (6 h)
- **Ficha:** 5.1.1 Low-code/no-code con IA: panorama, patrones y casos de uso (qué es y qué no es)

Palabras clave

low-code con IA, no-code con IA, automatización de procesos con IA, workflows con IA en empresa, integración de herramientas IA, automatización con Make y n8n, casos de uso IA en operaciones, orquestación de tareas con IA, IA para extracción y clasificación

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Explicar qué significa **low-code/no-code con IA** en un entorno empresarial y diferenciarlo de “hacer IA” (entrenar modelos) o de “usar ChatGPT suelto”.
2. Identificar los **patrones de automatización** más comunes: activación por evento, por programación y por consulta periódica (*polling*).
3. Reconocer los **casos de uso típicos** donde la IA aporta valor real dentro de un flujo: clasificar, extraer, resumir, etiquetar y redactar borradores.
4. Entender los **límites operativos** (errores, datos incompletos, calidad, costes, seguridad) y cómo se controlan con rutas de revisión y trazabilidad.

Síntesis

Low-code/no-code con IA significa **automatizar procesos** conectando sistemas (correo, formularios, CRM, helpdesk, drive, etc.) y añadiendo pasos de IA para tareas cognitivas concretas (clasificar, extraer, resumir, etiquetar). No es “magia”: la automatización solo funciona bien si el proceso está claro, los datos están definidos y existen controles (validación, fallbacks y revisión humana). En empresa, el valor aparece cuando reduces tiempo y errores en tareas repetitivas, con trazabilidad y métricas.

Contenido principal

1) Qué es low-code/no-code con IA (explicado para empresa)

No-code significa crear automatizaciones sin programar (o con mínima configuración), usando interfaces visuales: eliges un disparador, conectas acciones y defines reglas.

Low-code significa que puedes añadir algo de lógica o scripting ligero cuando hace falta, pero sin construir una aplicación completa “desde cero”.

Cuando añadimos **IA**, normalmente no estamos “entrenando un modelo”. Lo que hacemos es **usar IA como un paso dentro de un flujo**, por ejemplo:

- leer un texto y clasificarlo (tipo de incidencia),

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Fichas de contenido

- extraer datos estructurados (importe, fecha, cliente),
- resumir un documento o una conversación,
- redactar un borrador (respuesta inicial al cliente),
- etiquetar y enrutar (asignar a un equipo).

Idea clave: no-code/low-code con IA es **orquestración de procesos**. La IA es un componente dentro del proceso, no el proceso completo.

2) Qué NO es (para evitar errores típicos al empezar)

Es importante entender qué no estás haciendo, porque ahí nacen muchas expectativas irreales:

1. No es entrenar modelos de IA

No vas a crear un modelo desde cero. Usas servicios de IA ya existentes (por ejemplo, un modelo de lenguaje) mediante un conector o una llamada HTTP (una petición a una API).

2. No es “poner un chat” en la empresa y ya

Un chat suelto ayuda a personas, pero no automatiza procesos. Una automatización conecta sistemas y genera resultados medibles (tickets, registros, tablas, notificaciones).

3. No es automatización sin riesgos

Automatizar implica riesgos: errores de datos, respuestas incorrectas, acciones equivocadas. Por eso debes diseñar rutas de revisión y control.

4. No es “sin mantenimiento”

Los procesos cambian, las herramientas cambian, las políticas cambian. Una automatización requiere operación mínima: logs, alertas, revisión y ajustes.

3) Panorama general: qué hace una automatización en empresa (vista de extremo a extremo)

Una automatización típica tiene esta forma:

1. **Entrada:** llega un evento (un email, un formulario, un mensaje, un fichero).
2. **Procesamiento:** se normalizan datos (limpieza, formato).
3. **Decisiones:** se decide ruta (prioridad, tipo de caso, revisión).
4. **Acciones:** se crea un ticket, se actualiza un CRM, se envía un email, se guarda un fichero.
5. **Control:** se valida salida, se loguea, y se notifica si hay error.

La IA suele entrar en el paso 3 (decisión) o para transformar entrada en datos (extracción), o para generar un borrador (respuesta). Pero siempre debe quedar claro:

- qué entra,
- qué sale,
- y qué pasa si la IA falla.

4) Patrones de activación del flujo (cómo “se dispara” una automatización)

En automatización, **cómo empieza** el proceso es una decisión técnica y operativa.

4.1 Activación por evento

El flujo se ejecuta cuando ocurre algo:

- llega un formulario,
- se recibe un email,
- se crea un registro en un sistema,
- un usuario pulsa un botón (webhook).

Este patrón es el más “natural” porque responde en tiempo real y reduce esperas.

Ventajas: rápido, eficiente, ideal para atención y operaciones.

Riesgos: más casos simultáneos, más necesidad de control de errores.

4.2 Activación por programación (scheduler / cron)

El flujo se ejecuta a una hora fija o cada cierto tiempo:

- cada día a las 9:00,
- cada hora,
- cada 15 minutos.

Ventajas: estable, controlable, útil para tareas “batch” (por lotes).

Riesgos: si se acumulan muchos elementos, puede crecer el tiempo/coste; si falla, puedes perder el ciclo.

4.3 Polling (consulta periódica)

El sistema “pregunta” cada X minutos si hay novedades:

- “¿hay nuevos emails?”,
- “¿hay nuevos pedidos?”,
- “¿hay nuevos tickets?”.

Se usa cuando no hay eventos/webhooks disponibles.

Ventajas: funciona casi siempre, aunque no haya disparador nativo.

Limitaciones: menos eficiente (consulta aunque no haya cambios), puede duplicar trabajo si no se controla el estado.

Consejo práctico: si existe evento/webhook, suele ser mejor que polling.

5) Dónde aporta valor la IA dentro del flujo (papel realista)

La IA aporta valor en tareas que, sin IA, implican lectura y juicio humano repetitivo. Los usos más comunes (y más útiles) son:

5.1 Clasificar (decidir tipo y prioridad)

Ejemplo: llega un email de soporte y la IA decide:

- categoría (factura, devolución, incidencia técnica),
- prioridad (alta si hay “pago duplicado”, baja si es “consulta general”),
- y asignación a equipo.

Por qué funciona: la clasificación es repetitiva y se basa en texto.

5.2 Extraer (convertir texto a datos)

Ejemplo: de un email o PDF extraer:

- número de pedido,
- importe,
- fecha,

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Fichas de contenido

- nombre de empresa,
- motivo.

Por qué funciona: reduces tecleo y errores.

Control esencial: si un dato no aparece, la IA debe devolver “no indicado”, no inventarlo.

5.3 Resumir (convertir texto largo en acción)

Ejemplo: resumen de una conversación o un documento y extracción de:

- decisiones,
- acciones,
- riesgos,
- próximos pasos.

Valor real: acelera coordinación y reduce pérdidas de información.

5.4 Etiquetar y normalizar (calidad de datos)

Ejemplo: convertir respuestas libres en campos estándar (estado, motivo, tipo).

Esto es clave porque las automatizaciones necesitan datos consistentes.

5.5 Redactar borradores (no envío automático sin control)

Ejemplo: generar borrador de respuesta al cliente basado en plantilla y política.

Regla profesional: la IA puede redactar, pero el envío automático requiere controles, especialmente en casos sensibles.

6) Casos de uso típicos en empresa (ejemplos completos)

A continuación, una lista de casos típicos con IA “real” dentro de automatización:

1. Atención al cliente / soporte

Entrada: email/formulario → IA clasifica y extrae datos → crea ticket → propone borrador de respuesta → deriva si riesgo alto.

2. Administración / facturas

Entrada: factura (texto/PDF) → IA extrae campos → valida esquema → registra en tabla/sistema → alerta si falta dato crítico.

3. Ventas / leads

Entrada: lead web → IA clasifica sector e interés → enruta a comercial → genera resumen del lead y email interno.

4. RRHH / onboarding

Entrada: alta de empleado → genera checklist → envía recursos → crea tareas → responde FAQ internas con base documental (RAG).

5. Operaciones / incidencias

Entrada: aviso de incidencia → IA resume y categoriza → notifica al equipo → crea registro → solicita datos faltantes automáticamente.

7) Límites operativos (lo que puede fallar y cómo se diseña para que no sea un desastre)

La IA y la automatización fallan por razones muy concretas. Aquí están las principales y cómo mitigarlas:

7.1 Datos incompletos o sucios

Si el email no trae nº pedido, el flujo no puede completar.

Mitigación: ruta de “faltan datos” que pida lo mínimo y no avance hasta tenerlo.

7.2 IA que inventa (alucinaciones)

Riesgo típico si pides a un modelo “extrae campos” sin reglas.

Mitigación: contrato de salida estricto (campos obligatorios, “no indicado”), validación y fallback.

7.3 Errores técnicos (tiempos de espera, permisos, límites de API)

Puede fallar la conexión (401/403/429/5xx: códigos típicos de error).

- **401/403:** credenciales o permisos.
- **429:** demasiadas peticiones (rate limit).
- **5xx:** error del servicio.

Mitigación: reintentos controlados, ruta de error, alertas y logs.

7.4 Coste por ejecución (volumen)

Cada ejecución consume operaciones. Un flujo mal diseñado se dispara demasiado.

Mitigación: filtrar al inicio, procesar por lotes si procede, evitar polling excesivo.

8) Qué debes llevarte de esta ficha

- La IA no sustituye el diseño del proceso: lo complementa.
- Low-code/no-code con IA es “hacer que el trabajo suceda” entre sistemas con control.
- La calidad depende de datos definidos, validación y rutas de revisión/error.
- Un flujo profesional siempre tiene: entrada clara, salida definida, logs, KPIs y fallback.

Relación de ideas clave

- Low-code/no-code con IA = automatización de procesos + pasos de IA para tareas cognitivas concretas.
- Patrones de activación: evento, programación y polling (elige según canal y eficiencia).
- IA aporta valor en clasificar, extraer, resumir, etiquetar y redactar borradores.
- Límites operativos: datos incompletos, inventos de IA, errores técnicos, costes por volumen.
- Diseño profesional = validación, trazabilidad, rutas de revisión y error.

Más información en ChatGPT

- “Dame 10 casos de uso de automatización con IA para una pyme de [sector]. Para cada uno: entrada, paso de IA, validación, salida, riesgos y fallback.”
- “Explícame con ejemplos cuándo conviene evento vs programación vs polling en un flujo de automatización.”
- “Diseña un ‘contrato de salida’ para una IA que extrae datos de emails: campos obligatorios, valores permitidos y qué devolver si falta información.”

5.1.2 Datos en automatización: JSON estándar, campos, tipos, listas y validación básica

Referencia

- **Módulo:** 5. Herramientas low-code y no-code con IA (42 h)
- **Apartado:** 5.1. Introducción y Contexto Actual (6 h)
- **Ficha:** 5.1.2 Datos en automatización: JSON estándar, campos, tipos, listas y validación básica

Palabras clave

JSON estándar, datos en automatización, esquema de datos, campos y tipos, arrays listas JSON, normalización de datos, validación entrada salida, nulos, fechas ISO 8601, metadatos, contrato de datos

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Entender qué es **JSON** (*JavaScript Object Notation*) como formato estándar para “hacer viajar” datos entre herramientas.
2. Diferenciar **campos, tipos** (texto, número, booleano, fecha) y **listas** (arrays) y por qué esto afecta a la automatización.
3. Diseñar un **esquema estándar** (contrato de datos) para un proceso típico de empresa: qué campos mínimos, cuáles opcionales y cómo nombrarlos.
4. Aplicar **validación básica** de entrada y salida: detectar datos faltantes, tipos incorrectos y valores no permitidos, y activar ruta de revisión/error.
5. Normalizar datos frecuentes (texto, fechas, nulos) para evitar fallos y mantener trazabilidad.

Síntesis

En automatización, el 80% de los problemas no viene de “la herramienta”, sino de los **datos**: campos faltantes, formatos distintos, listas mal tratadas o valores inesperados. JSON es el formato estándar para mover información entre sistemas, pero funciona bien solo si defines un **contrato de datos**: qué campos deben existir, qué tipo tiene cada uno y cómo validarlos. Esta ficha te enseña a diseñar ese estándar y a aplicar validación y normalización básica, imprescindible para flujos con IA (extracción, clasificación y resumen).

Contenido principal

1) Por qué los datos son el núcleo de cualquier automatización

Una automatización es una cadena de pasos. Cada paso recibe datos y entrega datos. Si los datos no están bien definidos, pasan tres cosas:

- el flujo se rompe (error técnico),

- el flujo funciona pero hace algo incorrecto (error silencioso),
- o el flujo “parece” funcionar, pero genera retrabajo (calidad baja).

Por eso, antes de pensar en “qué módulo uso” (Make, n8n), debes dominar el lenguaje común de datos: **JSON**, campos, tipos y validación.

2) Qué es JSON (explicación clara y útil)

JSON (*JavaScript Object Notation*) es un formato de texto para representar información de forma estructurada. Se usa porque:

- es legible (puedes verlo y entenderlo),
- es estándar en APIs y herramientas,
- permite objetos (diccionarios) y listas (arrays).

Ejemplo simple (un “objeto” JSON):

```
{  
  "cliente": "Empresa X",  
  "pedido_id": "A12345",  
  "importe": 580.80,  
  "urgente": false  
}
```

Idea clave: en automatización, casi todo lo que entra/sale es JSON, aunque la herramienta te lo muestre “bonito”.

3) Campos: qué son y cómo nombrarlos bien

Un **campo** es una “columna” dentro del objeto: pedido_id, importe, etc.

3.1 Buenas prácticas de nombres

- Nombres consistentes y claros: pedido_id mejor que id, si hay varios IDs.
- Evita espacios y caracteres raros (para evitar errores).
- Usa un estilo consistente (por ejemplo snake_case o camelCase, pero siempre el mismo).

3.2 Campos mínimos vs opcionales

Define siempre:

- **campos mínimos:** si faltan, no se puede continuar (ej. pedido_id en una consulta de pedido).
- **campos opcionales:** mejoran el caso, pero no bloquean (ej. foto_url, comentarios).

Esta distinción es clave para diseñar rutas:

- si falta un campo mínimo → ruta de “datos faltantes” (pedir o revisar).
- si falta un opcional → continuar y marcar “no indicado”.

4) Tipos de datos: texto, número, booleano, fecha (y por qué importa)

Los **tipos** definen cómo se interpreta un campo:

- **Texto (string):** "Toledo", "A12345".
- **Número (number):** 580.80 (ojo con decimales).
- **Booleano (boolean):** true/false (sí/no real).
- **Fecha/hora:** normalmente texto con formato estándar (ver más abajo).

- **Objeto:** un JSON dentro de otro JSON.
- **Lista (array):** varios elementos (ver siguiente sección).

Por qué importa: muchas integraciones fallan porque un campo llega como texto cuando se esperaba número, o porque una fecha llega con formato distinto.

5) Listas (arrays): el punto donde se rompe mucha gente

Un **array** es una lista de elementos:

```
{
  "items": [
    {"producto": "A", "cantidad": 2},
    {"producto": "B", "cantidad": 1}
  ]
}
```

En automatización, las listas aparecen por todas partes:

- líneas de pedido,
- lista de tickets,
- resultados de una búsqueda,
- destinatarios,
- ficheros adjuntos.

5.1 Errores típicos con listas

- Tratar una lista como si fuera un único objeto (solo procesas el primer elemento).
- Olvidar que una lista puede venir vacía (y el flujo se rompe).
- Iterar sin control y disparar el coste/operaciones (procesar 500 elementos sin límite).

5.2 Estrategias prácticas para listas

- **Iteración controlada:** procesar cada ítem con un paso repetible.
- **Agregación:** reunir resultados en un solo resumen o tabla.
- **Batches (lotes):** procesar en grupos (por ejemplo, 50 en 50) cuando hay volumen.
- **Regla de seguridad:** si la lista supera X elementos, derivar a revisión o activar modo "batch".

6) Metadatos: datos sobre los datos (para trazabilidad)

Además de los campos "de negocio", conviene añadir **metadatos**, por ejemplo:

- `source` (de dónde viene: email/formulario/CRM),
- `timestamp` (cuándo se capturó),
- `trace_id` (identificador de ejecución),
- `version` (versión de esquema).

Esto ayuda a:

- depurar errores,
- auditar decisiones,
- y medir KPIs.

7) Normalización: convertir datos a un formato estándar

Normalizar es “limpiar” y homogeneizar. Es imprescindible porque los sistemas generan datos distintos.

7.1 Texto

- Recortar espacios,
- unificar mayúsculas/minúsculas si aplica,
- eliminar caracteres invisibles.

7.2 Fechas (formato estándar)

Un estándar común es **ISO 8601**, por ejemplo: 2026-03-06 o 2026-03-06T10:30:00.

Si la fecha llega como 06/03/2026, conviene convertirla.

Por qué: comparar fechas y filtrar por fecha depende del formato.

7.3 Nulos y “no indicado”

En automatización debes decidir cómo representar “no hay dato”:

- null (valor vacío real),
- o texto “no indicado” (más legible pero menos “técnico”).

Lo importante es ser consistente. Si mezclas “”, null y “N/A”, la validación se vuelve difícil.

8) Validación básica: entrada, salida y reglas de negocio

Validar significa comprobar que:

- los campos mínimos existen,
- los tipos son correctos,
- los valores son razonables.

8.1 Validación de entrada (antes de hacer nada)

Ejemplo: antes de crear un ticket:

- existe motivo,
- existe contacto,
- existe un identificador (pedido_id o alternativa).

Si falla, no sigas: activa ruta de “faltan datos”.

8.2 Validación de salida (después de un paso crítico)

Ejemplo: si la IA extrae datos:

- ¿devolvió todos los campos obligatorios?
- ¿devolvió números donde toca?
- ¿devolvió “no indicado” cuando no existe? (no inventar)

8.3 Validación de reglas de negocio

- importe > 0
- prioridad ∈ {alta, media, baja}
- estado ∈ {nuevo, en_proceso, cerrado}

Esto evita valores “inventados” o inconsistentes.

9) Plantilla de “contrato de datos” (esquema estándar mínimo)

Para un proceso típico (ej.: ticket de soporte), define:

- case_id (opcional, si ya existe)
- source (email/formulario/web)
- timestamp (ISO)
- customer_name (texto)
- customer_email (texto)
- topic (categoría)
- priority (alta/media/baja)
- summary (resumen breve)
- details (texto)
- attachments (lista)
- flags (objeto: needs_review, sensitive_data_detected)

Con este contrato, cualquier flujo se vuelve más robusto.

Relación de ideas clave

- JSON es el formato estándar para mover datos entre herramientas.
- Los fallos típicos vienen de campos faltantes, tipos incorrectos y listas mal tratadas.
- Define un contrato de datos: campos mínimos, opcionales, tipos y valores permitidos.
- Normaliza texto, fechas y nulos para evitar errores y mejorar trazabilidad.
- Valida entrada y salida, especialmente cuando hay IA en el flujo.

Más información en ChatGPT

- “Diseña un contrato de datos (JSON) para el caso: [caso]. Incluye campos mínimos, opcionales, tipos y valores permitidos.”
- “Dame reglas de validación para este JSON y ejemplos de errores típicos y cómo tratarlos: [pega JSON].”

5.1.3 Diseño del proceso operable: evento vs programación, rutas (principal/revisión/error) y trazabilidad/KPIs

Referencia

- **Módulo:** 5
- **Apartado:** 5.1. Introducción y Contexto Actual
- **Ficha:** 5.1.3 Diseño del proceso operable: evento vs programación, rutas (principal/revisión/error) y trazabilidad/KPIs

Palabras clave

diseño de workflow, evento vs batch, rutas de revisión, manejo de errores, trazabilidad, logging, KPIs automatización, observabilidad, alertas, ruta de fallback, depuración

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Decidir cuándo usar un flujo por **evento** y cuándo por **programación** (batch) según proceso y riesgo.
2. Diseñar un flujo con rutas claras: **principal, revisión y error técnico**, sin mezclar responsabilidades.
3. Definir **trazabilidad**: qué se registra (logs), con qué identificadores y para qué sirve.
4. Definir KPIs operativos (tiempo, error, retrabajo) y usar esos KPIs para mejorar el flujo.

Síntesis

Un flujo “operable” no es solo un diagrama bonito: es un proceso que se puede ejecutar miles de veces sin romperse y que se puede depurar cuando algo falla. Para ello necesitas decidir bien el disparador (evento o batch), diseñar rutas separadas (principal, revisión y error técnico), y añadir trazabilidad (logs + identificador de ejecución). Además, debes medir KPIs para demostrar valor y detectar degradación. Esta ficha te enseña el diseño mínimo profesional para automatizaciones con IA.

Contenido principal

1) Qué significa “operable” (y por qué es el estándar profesional)

Un flujo operable es aquel que:

- se ejecuta de forma repetible,
- falla de forma controlada (no en silencio),
- deja rastro (logs),
- y permite mejora continua.

Muchos flujos fallan porque solo se diseñan para “el caso perfecto”. En empresa, el caso perfecto es minoría: faltan datos, cambian formatos, hay picos de volumen y ocurren errores de permisos.

2) Elegir el disparador: evento vs programación (batch)

2.1 Evento (*en tiempo real*)

Se dispara cuando ocurre algo: email, formulario, webhook.

Cuándo conviene:

- atención al cliente,
- tickets,
- leads,
- incidencias operativas.

Ventajas: respuesta rápida, experiencia mejor.

Riesgos: picos de carga, más necesidad de control y alertas.

2.2 Programación / batch (*por lotes*)

Se ejecuta cada X tiempo y procesa un conjunto.

Cuándo conviene:

- resúmenes diarios,
- sincronizaciones,
- consolidación de datos,
- tareas de limpieza.

Ventajas: controlable, estable, fácil de planificar.

Riesgos: acumulación; si falla un ciclo, se pierde continuidad si no hay reintento.

Regla práctica de decisión:

Si el usuario espera respuesta inmediata → evento.

Si el objetivo es consolidar/reportar → batch.

3) Rutas del flujo: principal, revisión y error técnico

Separar rutas es lo que distingue un flujo amateur de uno profesional.

3.1 Ruta principal (*happy path*)

Aquí ocurre el caso “normal”: datos correctos, salida válida.

Debe ser limpia, sin “parches”. Si la ruta principal está llena de excepciones, el flujo será difícil de mantener.

3.2 Ruta de revisión (*calidad / incertidumbre / riesgo*)

Esta ruta se activa cuando:

- faltan datos mínimos,
- la IA devuelve salida dudosa,
- el caso es sensible (dinero, legal, reputación),
- o hay ambigüedad.

Ejemplo: IA clasifica un ticket como “reembolso” y detecta palabras como “denuncia” → revisión humana obligatoria.

La ruta de revisión suele:

- crear un ticket marcado “revisión”,
- notificar a un responsable,
- y adjuntar el resumen y los datos detectados.

3.3 Ruta de error técnico (fallo del sistema)

Errores típicos:

- permisos (401/403),
- límite de peticiones (429),
- fallo del servicio (5xx),
- timeouts.

Esta ruta debe:

- registrar el error,
- reintentar si procede (con límites),
- y alertar si persiste.

Regla: el error técnico no debe mezclarse con revisión de negocio.

4) Trazabilidad: logs y “trace_id” (identificador de ejecución)

Trazabilidad significa que puedes responder:

- qué pasó,
- cuándo,
- con qué datos,
- y en qué paso falló.

4.1 Qué registrar mínimo

- trace_id (id único por ejecución),
- source (origen),
- timestamp inicio/fin,
- resultado (OK/revisión/error),
- mensaje de error si aplica,
- resumen de entrada/salida (sin datos sensibles innecesarios).

4.2 Por qué esto importa

Sin logs:

- no puedes depurar,
- no puedes medir,
- no puedes demostrar valor.

Con logs:

- detectas patrones (“fallan los martes por permisos”),
- mejoras procesos,
- y justificas decisiones.

5) KPIs operativos (los que de verdad sirven)

Define 4–6 KPIs simples:

- **Tiempo de ciclo** (desde entrada hasta salida).
- **Tasa de éxito** (OK / total).

- **Tasa de revisión** (cuántos casos requieren humano).
- **Tasa de error técnico** (y causa).
- **Retrabajo** (casos que vuelven por error).
- **Volumen** (para dimensionar y controlar coste).

Interpretación importante:

- Una tasa de revisión muy baja puede ser mala si la IA está “pasando casos sensibles” sin control.
- Una tasa de revisión muy alta indica que el flujo no está bien definido o la IA no tiene contrato de salida.

6) Diseño “anti-sorpresas”: validación y fallbacks

Para que el flujo sea operable, incluye:

- validación de entrada (antes de gastar recursos),
- contrato de salida para IA (campos obligatorios),
- fallback: reintento limitado o derivación a revisión.

Esto se conecta con 5.1.2 (validación de datos) y 5.5.x (calidad y producción).

7) Plantilla de diseño de flujo operable (para alumnos)

- Disparador: evento / batch
- Entrada: fuente y campos mínimos
- Ruta principal: pasos y salida
- Ruta revisión: triggers, qué se pide, quién revisa
- Ruta error técnico: reintentos, alertas, logs
- Trazabilidad: trace_id, campos logueados
- KPIs: lista y cómo medirlos

Relación de ideas clave

- Elegir bien evento vs batch define experiencia y estabilidad.
- Separar rutas (principal/revisión/error) hace el flujo mantenible.
- Trazabilidad (logs + trace_id) es obligatoria para depurar y medir.
- KPIs deben medir calidad y operación, no solo volumen.
- Validación + fallback evitan errores silenciosos.

Más información en ChatGPT

- “Diseña un flujo operable para [caso] con rutas principal/revisión/error, trazabilidad mínima y KPIs.”
- “Dame un set de triggers de revisión (casos sensibles, incertidumbre, datos incompletos) y cómo tratarlos sin romper el flujo.”

5.1.4 Diseño de procesos automatizables: selección, requisitos y mapa AS-IS → TO-BE

Referencia

- **Módulo:** 5
- **Apartado:** 5.1. Introducción y Contexto Actual
- **Ficha:** 5.1.4 Diseño de procesos automatizables: selección, requisitos y mapa AS-IS → TO-BE

Palabras clave

selección proceso automatizable, mapa de procesos, AS-IS TO-BE, requisitos automatización, valor vs riesgo, complejidad de proceso, alcance incluye excluye, definición de datos, preparación de piloto

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Seleccionar un proceso “troncal” automatizable usando criterios de **valor, riesgo, datos y complejidad**.
2. Definir requisitos del proceso: alcance, entradas/salidas, reglas y excepciones, responsables y puntos de control humano.
3. Construir un mapa **AS-IS** (cómo se hace hoy) y un mapa **TO-BE** (cómo se hará con automatización) de forma clara y operativa.
4. Identificar qué información falta para automatizar (datos, documentación, integraciones) y convertirlo en un plan de acción.

Síntesis

No todos los procesos son buenos candidatos para automatizar. Un proceso automatizable es repetitivo, tiene datos disponibles, reglas claras y un riesgo controlable. Para elegir bien se usa una matriz simple (valor vs riesgo) y se describe el proceso con mapas **AS-IS** (situación actual) y **TO-BE** (situación futura). Esta ficha te enseña a seleccionar un caso viable y a traducirlo a un diseño que una herramienta low-code/no-code puede implementar con rutas de revisión, control humano y métricas.

Contenido principal

1) Qué significa AS-IS y TO-BE (explicación sin jerga)

- **AS-IS** significa “como es ahora”: cómo se hace hoy el proceso, con personas, herramientas y problemas reales.
- **TO-BE** significa “como debería ser”: cómo quedará el proceso tras la automatización, con pasos nuevos, controles y resultados medibles.

El objetivo no es dibujar bonito: es entender dónde se pierde tiempo, dónde hay errores y qué parte se puede automatizar sin romper la operación.

2) Criterios para elegir un proceso automatizable (matriz práctica)

Para seleccionar un buen “caso troncal”, evalúa:

2.1 Valor (*impacto*)

- ¿Cuánto tiempo consume al mes?
- ¿Cuántos casos hay (volumen)?
- ¿Qué duele (retrasos, errores, clientes molestos)?
- ¿Qué KPI mejorarías?

2.2 Riesgo (*si falla, qué pasa*)

- ¿Afecta a dinero, legal, reputación?
- ¿Implica datos sensibles?
- ¿Puede generar acciones irreversibles?

Procesos de alto riesgo pueden automatizarse parcialmente, pero necesitan revisión humana.

2.3 Datos disponibles (*la condición “real”*)

- ¿Existen campos claros? (pedido_id, email, fecha)
- ¿Hay una fuente de verdad? (CRM, ERP, hoja)
- ¿Hay documentación de reglas y excepciones?

Si los datos no existen o están desordenados, primero hay que arreglar esa base.

2.4 Complejidad (*excepciones y variabilidad*)

- ¿El proceso tiene 10 excepciones distintas?
- ¿Depende de decisiones humanas subjetivas?
- ¿Cambia cada semana?

Cuanta más variabilidad, más difícil automatizar sin fricción. A veces conviene automatizar solo el triage (clasificación y registro) y dejar decisión a humano.

3) Definir requisitos: lo que debe quedar escrito antes de automatizar

Un proceso automatizable debe tener:

- **Entrada:** de dónde llega (email, formulario, sistema).
- **Salida:** qué se genera (ticket, registro, email, tabla).
- **Reglas:** cómo se decide prioridad y ruta.
- **Excepciones:** qué pasa si falta dato o hay caso sensible.
- **Control humano:** dónde se revisa (revisión/approval).
- **KPIs:** cómo medir éxito.

Esto es el “brief” del proceso. Sin esto, la automatización será un conjunto de módulos sin sentido.

4) Cómo construir un mapa AS-IS (paso a paso)

El mapa AS-IS debe incluir:

1. Quién inicia el proceso (rol).

2. Qué herramientas usa (correo, Excel, CRM).
3. Qué decisiones se toman (y con qué criterio).
4. Qué salidas se generan.
5. Dónde se pierde tiempo (esperas, copias manuales).
6. Dónde hay errores (datos mal copiados, olvidos).

Ejemplo típico AS-IS (soporte):

- llega email → persona lee → decide categoría → copia datos a ticket → responde → si falta info, vuelve a preguntar.

5) Cómo construir un mapa TO-BE (automatizado) con control

El mapa TO-BE define qué se automatiza y qué no.

Ejemplo TO-BE (soporte):

- llega email → se normaliza → IA clasifica y extrae → se valida contrato de datos →
 - si OK: crea ticket + borrador de respuesta
 - si faltan datos: ruta revisión/pedir datos
 - si sensible: derivación humana
- se registran logs y KPIs.

Clave: TO-BE debe incluir rutas (principal/revisión/error) y trazabilidad (trace_id).

6) Alcance: incluye/excluye (evitar el “quiero todo”)

Define explícitamente:

- qué incluye (3–5 tareas),
- qué excluye (casos especiales, excepciones),
- qué queda para fase 2.

Esto reduce riesgo y acelera resultados.

7) Entregable final recomendado (para alumno)

Para presentar un caso troncal, entrega:

- 1 párrafo de contexto y objetivo (KPI)
- Matriz valor/riesgo (simple)
- Mapa AS-IS (pasos)
- Mapa TO-BE (rutas)
- Datos mínimos y contrato JSON (campos)
- Reglas de revisión/error y escalado
- KPIs y plan de piloto (2–4 semanas)

Relación de ideas clave

- AS-IS y TO-BE son herramientas para diseñar automatización con realismo.
- Selecciona procesos por valor, riesgo, datos disponibles y complejidad.
- Define requisitos antes de construir: entrada, salida, reglas, excepciones, control humano y KPIs.

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Fichas de contenido

- El TO-BE debe incluir rutas y trazabilidad.
- Un buen alcance inicial evita el “quiero todo” y permite pilotar.

Más información en ChatGPT

- “Ayúdame a elegir un proceso automatizable en una empresa de [sector]. Dame 5 candidatos y evalúalos por valor, riesgo, datos y complejidad.”
- “Convierte este proceso AS-IS en un TO-BE automatizable con rutas principal/revisión/error y KPIs: [describe proceso].”

5.2.1 Marco de decisión: cuándo elegir Make o n8n (criterios, pesos y trade-offs)

Referencia

- **Módulo:** 5. Herramientas low-code y no-code con IA (42 h)
- **Apartado:** 5.2. Estrategia y Comparativa: Make vs n8n (6 h)
- **Ficha:** 5.2.1 Marco de decisión: cuándo elegir Make o n8n (criterios, pesos y trade-offs)

Palabras clave

Make vs n8n, elegir herramienta automatización, criterios decisión workflow, trade-offs low-code, coste por operación, self-hosted n8n, gobernanza automatizaciones, observabilidad logs, conectores, HTTP API, escalabilidad

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Usar un **marco de decisión** para escoger Make o n8n según criterios técnicos y operativos (no por preferencias).
2. Definir **pesos** por criterio (qué importa más en tu caso) y justificar la elección con argumentos claros.
3. Identificar **trade-offs** (lo que ganas y lo que pierdes) en coste, control, gobierno, integración y mantenimiento.
4. Aplicar el marco a un caso real: escoger herramienta para un flujo con IA, validación de datos, rutas de revisión/error y trazabilidad.

Síntesis

Elegir entre Make y n8n no es una discusión de “cuál es mejor”, sino de **cuál encaja mejor con tu caso**. La decisión se toma evaluando criterios como: tipo de flujo (evento/batch), tratamiento de datos y listas, conectividad HTTP/API, observabilidad (logs), gobierno de entornos (dev/test/prod), volumen y coste, seguridad y posibilidad de alojar en tu infraestructura (self-hosted). Esta ficha te enseña un método con pesos y trade-offs para tomar una decisión defendible ante un cliente o dirección.

Contenido principal

1) Por qué necesitas un marco de decisión (y por qué “me gusta más” no vale)

En empresa, una herramienta se elige por:

- **encaje con el proceso**,
- **capacidad de operar a escala** (miles de ejecuciones sin drama),
- **coste total** (licencias + mantenimiento + retraining),
- **seguridad y gobierno**,
- **capacidad de evolucionar** (integraciones nuevas, cambios).

Un marco de decisión evita errores típicos:

- elegir por moda y descubrir límites al crecer,
- elegir por coste inicial y pagar caro en mantenimiento,
- elegir por “conectores” y fallar por falta de control/observabilidad.

2) Qué es Make y qué es n8n (definición rápida orientada a decisión)

- **Make**: plataforma cloud con interfaz visual muy orientada a rapidez, conectores y productividad inmediata. Suele destacar en “montar rápido” y trabajar con conectores listos.
- **n8n**: plataforma de automatización con enfoque muy flexible, donde “workflow” y lógica pueden ser más controlables y extensibles; además permite **self-hosting** (alojarlo en tu propia infraestructura), lo que impacta seguridad, control y costes.

Importante: ambos pueden hacer casi lo mismo en teoría. La diferencia está en **operar, gobernar, extender y mantener**.

3) Los criterios de decisión (qué comparar de verdad)

A continuación tienes criterios “de producción”, no de demo.

3.1 Tipo de activación y diseño del flujo (evento, programación, polling)

- Si necesitas flujos “muy en tiempo real”, ambos pueden, pero importa cómo gestionan reintentos, límites y errores.
- Si haces procesamiento por lotes, importa la facilidad de programar, paginar, controlar volumen y no duplicar.

Preguntas clave

- ¿Tu flujo es sensible a latencia (segundos/minutos) o vale por lotes?
- ¿Habrá picos de volumen?
- ¿Necesitas control fino de reintentos y colas?

3.2 Datos y listas (arrays) + transformación

En automatización real hay listas: líneas de pedido, tickets, adjuntos, resultados. Aquí se decide si la herramienta te permite trabajar cómodo con:

- iteraciones controladas,
- agregaciones,
- paginación,
- validación de tipos.

Señal de riesgo: si tu caso tiene muchas listas y campos, el coste de mantenimiento sube si la herramienta no es cómoda para “modelar datos”.

3.3 HTTP, APIs y conectividad universal

Tarde o temprano necesitarás:

- llamar a una API por HTTP,
- pasar headers (cabeceras), tokens, paginación,
- parsear JSON,
- manejar errores 401/403/429/5xx,
- reintentar con backoff (esperas).

Si tu empresa integra mucho, este criterio pesa mucho.

3.4 Observabilidad (*logs, trazabilidad, debugging*)

Observabilidad significa poder responder:

- qué ejecución falló,
- en qué paso,
- con qué datos,
- y qué reintentos se hicieron.

Esto se traduce en:

- logs legibles,
- identificador de ejecución (*trace_id*),
- métricas,
- alertas.

En flujos con IA, la observabilidad es crítica: debes poder auditar por qué se clasificó algo o por qué faltó un campo.

3.5 Gobierno/entornos (*dev-test-prod*) y control de cambios

En empresa, necesitas:

- trabajar en un entorno de pruebas,
- pasar a producción con control,
- versionar,
- documentar cambios,
- limitar quién puede editar.

Si no, el flujo “se rompe” por cambios improvisados.

3.6 Volumen y coste (*TCO: coste total de propiedad*)

No es solo el precio del plan: es el coste por operación, por ejecución y por mantenimiento.

TCO incluye:

- licencias,
- tiempo humano,
- retrabajo por fallos,
- coste de caídas/errores,
- coste de escalar.

Un flujo barato pero frágil puede salir caro.

3.7 Seguridad, privacidad y hosting

- ¿Puedes usar cloud sin problema?
- ¿Necesitas alojar datos dentro de tu infraestructura? (requisito frecuente en ciertos sectores)
- ¿Necesitas control de accesos por rol y auditoría?

Aquí suele destacar la posibilidad de **self-hosting** de n8n cuando hay requisitos estrictos (sin afirmar que sea siempre obligatorio).

4) Pesos: cómo convertir criterios en una decisión defendible

Un marco de decisión serio no dice “todo importa igual”. Ponderas.

Ejemplo de pesos (orientativo):

- Seguridad/hosting: 20%
- Observabilidad: 15%
- Datos/listas: 15%
- HTTP/APIs: 15%
- Gobierno/entornos: 15%
- Coste/volumen: 10%
- Time-to-value (rapidez de montaje): 10%

Cómo se usa:

1. puntúas cada criterio (1–5) para Make y n8n en TU caso,
2. multiplicas por el peso,
3. justificas el resultado con riesgos y mitigaciones.

Esto es lo que permite “defender” la elección ante cliente/dirección.

5) Trade-offs típicos (lo que ganas y lo que pierdes)

No hay elección perfecta. Debes decir explícitamente:

- “elijo X porque..., pero acepto Y”.

Ejemplos de trade-offs frecuentes:

- **Rapidez inicial vs control profundo:** montar rápido vs gobernar y extender a largo plazo.
- **Cloud conveniencia vs requisitos de datos:** facilidad vs restricciones de privacidad/hosting.
- **Menos configuración vs más flexibilidad:** facilidad vs capacidad de personalizar.

6) Plantilla final de decisión (entregable profesional)

Cuando el alumno tenga que justificar, usa esta plantilla:

- Caso y objetivos (KPI)
- Requisitos (canal, volumen, datos, riesgos)
- Criterios + pesos
- Puntuación Make vs n8n
- Riesgos principales por herramienta
- Mitigaciones
- Decisión final + por qué (3 razones)
- Qué monitorizar en producción (KPIs + alertas)

Relación de ideas clave

- La decisión Make vs n8n se toma por criterios y pesos, no por gustos.
- Los criterios clave son: datos/listas, HTTP/APIs, observabilidad, gobierno, coste/volumen, seguridad/hosting.
- Explicitar trade-offs es parte de la decisión profesional.
- La elección se defiende con matriz + riesgos + mitigaciones.

Más información en ChatGPT

- “Construye una matriz de decisión Make vs n8n para este caso: [describe caso], con criterios ponderados, puntuación y recomendación final.”
- “Dame una lista de riesgos y mitigaciones al elegir Make (o n8n) para un flujo con IA, datos sensibles y alto volumen.”

5.2.2 Modelo mental y terminología comparada (Make/n8n): equivalencias funcionales

Referencia

- **Módulo:** 5
- **Apartado:** 5.2. Estrategia y Comparativa: Make vs n8n
- **Ficha:** 5.2.2 Modelo mental y terminología comparada (Make/n8n): equivalencias funcionales

Palabras clave

terminología Make n8n, equivalencias escenario workflow, module node, router if, bundle item, mapping, variables, error handler, trigger webhook scheduler, iteración arrays, agregación

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Traducir el mismo flujo entre Make y n8n sin cambiar su lógica (pensar en equivalencias).
2. Entender términos clave: **escenario/workflow, module/node, bundle/item, router/if, trigger, error handler, mapping**.
3. Reconocer cómo se representan datos y listas en cada herramienta, y cómo afecta al diseño (iteración, agregación, validación).
4. Evitar errores típicos al “copiar” un flujo de una herramienta a otra.

Síntesis

Make y n8n pueden resolver problemas similares, pero usan vocabulario y conceptos distintos. Si no dominas equivalencias, te pierdes: crees que “no existe” algo cuando simplemente se llama diferente. Esta ficha es un diccionario práctico: te permite mapear conceptos y operar con el mismo modelo mental: disparador → pasos → decisiones → manejo de errores → salida. Es esencial para comparar herramientas y para trabajar en equipos donde conviven ambas.

Contenido principal

1) Por qué importa la terminología (y por qué no es “solo palabras”)

En automatización, el vocabulario define cómo piensas:

- si entiendes qué es una “unidad de ejecución” (un item/bundle),
- si entiendes cómo se ramifica (router/if),
- si entiendes cómo se manejan errores, puedes diseñar flujos robustos.

Cuando cambias de herramienta, el error típico es “trasladar el flujo” sin adaptar el concepto de datos o de errores.

2) Equivalencias principales (modelo mental común)

Piensa siempre en 6 piezas:

1. **Contenedor del flujo**
 - Make: *Scenario*
 - n8n: *Workflow*
Es "el proceso" completo.
2. **Paso ejecutable**
 - Make: *Module*
 - n8n: *Node*
Es una acción: leer email, HTTP request, escribir en sheets, etc.
3. **Disparador** (cómo empieza)
 - Make: módulo trigger (Scheduler/Webhook/Watch...)
 - n8n: trigger node (Cron/Webhook/Trigger...)
Es el "evento" inicial.
4. **Unidad de dato procesada**
 - Make: *Bundle* (unidad que pasa por módulos)
 - n8n: *Item* (unidad que pasa por nodos)
Esto es crucial: muchas confusiones vienen de aquí.
5. **Ramas/decisiones**
 - Make: *Router* + filtros
 - n8n: *IF / Switch* / rutas según condiciones
Esto define qué camino sigue el flujo.
6. **Errores y reintentos**
 - Make: *Error handlers* (manejadores)
 - n8n: manejo de errores + nodos de captura/estrategias (según diseño)
Aquí se define si el flujo falla "con control" o "rompe".

3) Bundles vs Items (la equivalencia que más falla)

Esto es crítico para listas y volumen.

- En Make, un módulo puede emitir varios **bundles**: por ejemplo, "buscar emails" devuelve un bundle por email.
- En n8n, un nodo devuelve una lista de **items**: cada item es un registro.

Implicación práctica:

Cuando un paso devuelve 100 resultados, tienes 100 unidades que podrían disparar acciones posteriores (crear 100 tickets). Debes controlarlo:

- filtrando,
- limitando,
- agregando,
- o procesando por lotes.

4) Mapping (mapeo) y expresiones: “cómo conectas campos”

En ambas herramientas conectar datos significa:

- tomar campos de salida de un paso,
- transformarlos,
- y asignarlos a entradas del siguiente paso.

En Make se habla mucho de **mapping** (mapeo). En n8n se usan expresiones y referencias a campos del item.

Lo esencial: definir un “contrato de datos” (ver 5.1.2) y mapear consistentemente.

5) Router / IF y diseño de rutas (principal, revisión, error)

La equivalencia no es solo técnica, es de arquitectura:

- Ruta principal: casos normales.
- Ruta de revisión: datos faltantes, caso sensible, incertidumbre IA.
- Ruta de error técnico: permisos, timeouts, fallos API.

Tu “modelo mental” debe seguir existiendo aunque cambie el nombre de los componentes.

6) Error handler / manejo de errores: entender lo mínimo

Un flujo profesional no solo “falla”:

- registra,
- reintenta con límite,
- alerta,
- deriva a revisión.

En Make esto suele formalizarse con “error handlers”. En n8n se diseña con nodos y rutas.

Lo importante es el resultado: **operabilidad**.

7) Diccionario resumido (para usar como chuleta)

- Scenario = Workflow
- Module = Node
- Bundle = Item
- Router/Filters = IF/Switch/Conditions
- Scheduler = Cron
- Webhook = Webhook
- Error handler = ruta de error controlada
- Mapping = asignación/expresiones de campos

Relación de ideas clave

- Las herramientas difieren en vocabulario, pero comparten el mismo modelo mental de automatización.
- La equivalencia bundle/item es clave para listas y volumen.

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Fichas de contenido

- Mapping es “contrato de datos”: si no defines campos/tipos, el flujo se rompe.
- Router/IF y manejo de errores son arquitectura, no botones.

Más información en ChatGPT

- “Traduce este diseño de flujo (pasos + rutas) de Make a n8n manteniendo la misma lógica: [describe pasos].”
- “Explícame bundle vs item con un ejemplo de 100 emails y cómo evitar crear 100 tickets por error.”

5.2.3 Comparativa estructurada + taller de defensa por casos (argumentario)

Referencia

- **Módulo:** 5
- **Apartado:** 5.2. Estrategia y Comparativa: Make vs n8n
- **Ficha:** 5.2.3 Comparativa estructurada + taller de defensa por casos (argumentario)

Palabras clave

comparativa Make n8n, argumentario decisión herramienta, tabla criterios implicaciones, defender elección, riesgos y mitigación, caso troncal automatización, casos internos externos, taller de presentación

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Construir una **comparativa estructurada** (criterio → implicación → decisión → riesgos → mitigación) para Make vs n8n.
2. Elaborar un **argumentario** para defender una elección ante un cliente, equipo o dirección.
3. Aplicar la comparativa a 3 escenarios: **caso interno, caso externo y caso troncal** del módulo.
4. Anticipar preguntas difíciles (coste, seguridad, mantenimiento) y responder con evidencia y trade-offs.

Síntesis

Comparar herramientas no es listar “features”. Una comparativa útil conecta criterios con consecuencias reales: coste, control, mantenibilidad, seguridad y velocidad de entrega. Esta ficha propone un formato de tabla que obliga a pensar como profesional: cada criterio debe traducirse en una decisión y en un plan de mitigación. Además, incluye una dinámica de taller para que el alumno practique la defensa oral de su elección (como si presentara a un cliente).

Contenido principal

1) Por qué una comparativa “de features” es insuficiente

“Make tiene X conectores” o “n8n es self-hosted” son datos, pero no decisiones. La dirección o un cliente quiere saber:

- qué impacto tendrá en coste y tiempos,
- qué riesgos reduce o introduce,
- qué mantenimiento exige,

- y cómo se operará en producción.

Por eso, la comparativa debe ser **implicacional**: qué significa para tu caso.

2) Estructura de comparativa profesional (la tabla que debes usar)

Usa esta estructura:

Criterio → Implicación real → Decisión → Riesgo → Mitigación → Evidencia/KPI

Ejemplo (simplificado):

- Criterio: alto volumen
- Implicación: coste por ejecución + riesgo de saturación
- Decisión: elegir herramienta con mejor control/coste para volumen
- Riesgo: sobrecoste o fallos
- Mitigación: batch, filtrado, límites, alertas
- Evidencia: KPI de coste/mes y tasa de error

Esto transforma la comparativa en un documento defendible.

3) Criterios recomendados para comparar (lista “core”)

Incluye al menos:

1. Time-to-value (rapidez de montaje)
2. Datos y listas (iteración/agregación/validación)
3. HTTP/APIs y personalización
4. Observabilidad (logs, debugging)
5. Gobierno/entornos (dev/test/prod, versionado)
6. Seguridad/hosting (cloud vs self-host)
7. Coste total (TCO) con volumen
8. Mantenimiento y operación (quién lo sostiene)

4) Taller de defensa por casos (cómo practicar)

El objetivo es que el alumno pueda presentar su elección con claridad.

4.1 Caso interno (ejemplo típico)

- Soporte interno con base documental (procedimientos IT/RRHH)
- Riesgo: datos internos y permisos
- Clave: gobierno, permisos, RAG, trazabilidad

4.2 Caso externo (ejemplo típico)

- Atención al cliente + tickets + borrador de respuesta
- Riesgo: reputación, promesas, datos personales
- Clave: rutas de revisión, escalado, logs y control de datos

4.3 Caso troncal del módulo

- El proceso automatizable elegido en 5.1.4
- Clave: volumen, datos, validación, coste y operación

5) Preguntas difíciles y cómo responder (argumentario base)

Cuando presentes, te preguntarán:

- **¿Cuánto cuesta de verdad?**
Responde con TCO: licencias + operación + retraining + riesgo.
- **¿Qué pasa si falla?**
Responde con rutas: principal/revisión/error + alertas + escalado.
- **¿Cómo evitamos inventos de IA?**
Responde con contrato de salida, validación, y revisión humana en casos sensibles.
- **¿Quién lo mantiene?**
Responde con roles mínimos y calendario de revisión.
- **¿Podemos migrar si cambia la estrategia?**
Responde con diseño agnóstico: contrato de datos + flujo por patrones + documentación.

6) Plantilla final (para entregar en el aula)

Entrega:

1. Resumen del caso (1 párrafo)
2. Tabla comparativa con 8 criterios mínimos
3. Elección + 3 razones + trade-offs
4. Riesgos + mitigación
5. KPIs de piloto (2–6 semanas)

Relación de ideas clave

- Una comparativa útil conecta criterio con implicación real y plan de mitigación.
- La tabla “criterio→implicación→decisión→riesgo→mitigación” hace la elección defendible.
- Debes practicar la defensa en casos interno/externo/troncal.
- Las preguntas difíciles se responden con evidencias (KPIs, logs, rutas) y trade-offs explícitos.

Más información en ChatGPT

- “Crea una tabla comparativa Make vs n8n para este caso [describe], usando columnas: criterio, implicación, decisión, riesgos, mitigación, KPI.”
- “Prepárame un argumentario de 2 minutos para defender la elección de Make (o n8n) ante dirección, incluyendo trade-offs y plan de mitigación.”

5.3.1 [Make] Fundamentos y montaje del escenario: escenarios, módulos y disparadores (Scheduler/Webhook/Watch)

Referencia

- **Módulo:** 5. Herramientas low-code y no-code con IA (42 h)
- **Apartado:** 5.3. Make: construcción de automatizaciones (12 h)
- **Ficha:** 5.3.1 [Make] Fundamentos y montaje del escenario: escenarios, módulos y disparadores (Scheduler/Webhook/Watch)

Palabras clave

Make escenarios, Make módulos, triggers Make, Scheduler Make, Webhook Make, Watch module Make, primer escenario Make, ejecución reproducible, bundles Make, pruebas automatización, control de versiones

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Explicar qué es **Make** y cómo se estructura un **escenario** (scenario), un **módulo** (module) y la unidad de dato (**bundle**).
2. Elegir y configurar correctamente el **disparador** (*trigger*) según el tipo de proceso: **Scheduler**, **Webhook** o **Watch**.
3. Montar un primer escenario **verificable y reproducible**, con salida controlada, validación mínima y evidencia de ejecución.
4. Evitar errores típicos de principiantes: disparadores mal elegidos, ejecuciones duplicadas, falta de control de datos y pruebas sin trazabilidad.

Síntesis

Make es una plataforma cloud de automatización visual. En Make, un **escenario** es un flujo completo compuesto por **módulos** conectados. Un escenario siempre empieza con un **disparador** (trigger) que decide cuándo se ejecuta: por programación (**Scheduler**), por evento recibido (**Webhook**) o “vigilando” cambios en un sistema (**Watch**). El éxito del primer escenario no es “que funcione una vez”, sino que sea **reproducible**: misma entrada → misma salida esperada, con registros y control de datos.

Contenido principal

1) Qué es Make y cómo se organiza (modelo mental mínimo)

Make es una herramienta de automatización visual en la nube (cloud). Permite conectar aplicaciones y servicios mediante un flujo.

- **Escenario (Scenario):** el flujo completo (tu “proceso” automatizado).

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Fichas de contenido

- **Módulo (Module):** cada paso del escenario (leer email, hacer una petición HTTP, crear un ticket, etc.).
- **Bundle:** unidad de datos que “circula” entre módulos. Si un módulo devuelve 10 resultados, Make puede generar 10 bundles.

Consejo práctico: piensa en un escenario como una cadena de montaje. Los módulos son estaciones. Los bundles son “piezas” que pasan por la cadena.

2) Antes de construir: define el objetivo y el “resultado verificable”

Un error habitual es empezar a arrastrar módulos sin saber qué quieres demostrar.

Antes de tocar Make, escribe en una frase:

- **Entrada:** qué llega y desde dónde (email, formulario, webhook).
- **Transformación:** qué haces (clasificar, extraer, registrar).
- **Salida:** qué debe quedar creado (ticket, fila en sheet, email de aviso).

Resultado verificable significa que puedes comprobarlo:

- existe el ticket,
- existe el registro,
- o existe el archivo.

Recomendación: para el primer escenario, elige una salida simple y visible (por ejemplo, “crear una fila en Google Sheets” o “enviar un email a ti mismo”). Primero controlas el flujo; luego ya conectas sistemas críticos.

3) Disparadores (triggers): elegir bien es la mitad del proyecto

Un **trigger** es el módulo inicial que decide cuándo se ejecuta el escenario.

3.1 Scheduler (programación)

Scheduler ejecuta el escenario en un horario: cada X minutos, cada hora, cada día, etc.

Cuándo usarlo:

- resúmenes diarios,
- sincronizaciones por lotes,
- limpieza de datos,
- procesos que no necesitan tiempo real.

Riesgo típico: duplicar trabajo si no controlas “qué ya procesé”.

Consejo: guarda un “marcador” (última fecha procesada, último ID) para no repetir.

3.2 Webhook (evento recibido)

Un **webhook** es un “endpoint” (una URL) que recibe datos cuando ocurre un evento. En vez de “preguntar”, te “avisan”.

Cuándo usarlo:

- formularios web,
- apps que envían eventos,
- integraciones personalizadas,
- disparos desde otros sistemas.

Ventajas: rápido, eficiente, ideal para atención al cliente.

Riesgos: duplicados si el emisor reintenta; entradas mal formadas.

Consejos esenciales:

- valida entrada (campos mínimos) al inicio,
- añade un identificador de idempotencia (por ejemplo, `event_id`) si es posible,
- y registra `trace_id` para depurar.

3.3 Watch (vigilar cambios)

Los módulos **Watch** ("watch emails", "watch records", etc.) revisan periódicamente si hay nuevos elementos en una app. Es un patrón *polling* (consulta periódica).

Cuándo usarlo:

- cuando la app no ofrece webhook,
- cuando es suficiente revisar cada X tiempo.

Riesgo: consume operaciones y puede duplicar si no configura "desde cuándo" o "solo nuevos".

Consejo: configura siempre el modo "solo nuevos" o "desde el último", y prueba con pocos elementos.

4) Primer montaje del escenario (pasos recomendados)

Aquí tienes un método seguro para que el primer escenario sea reproducible.

Paso 1: Monta el trigger con "entrada controlada"

- Si es **Scheduler**, empieza con una tarea de ejemplo (un texto fijo) para probar cadena.
- Si es **Webhook**, envía un payload de prueba conocido.
- Si es **Watch**, limita a 1-3 elementos.

Recomendación: evita empezar con 200 emails o 500 registros. Primero valida el diseño.

Paso 2: Añade un módulo de "salida visible"

Ejemplos de salida:

- "Create a row" en una tabla,
- "Send an email",
- "Create a file".

El objetivo es ver el resultado sin ambigüedad.

Paso 3: Añade un módulo de "log" o registro mínimo

En Make, la trazabilidad se apoya en:

- nombres claros de módulos,
- y revisar el historial de ejecuciones.

Consejo: incluye un campo `trace_id` (por ejemplo, timestamp + origen) en la salida, para poder rastrear qué ejecución generó qué registro.

Paso 4: Ejecuta en modo prueba y revisa bundles

Antes de "activar" el escenario, ejecuta manualmente y revisa:

- qué campos entran,
- qué campos salen,
- cuántos bundles se generan.

Error típico: creer que llega un único dato y descubrir que llegan 50 bundles.

5) Buenas prácticas del primer día (las que evitan el 80% de problemas)

5.1 Nombra módulos y rutas con intención

En vez de "Module 1", "Module 2", usa:

- "Trigger: Webhook lead"
- "Validación: campos mínimos"
- "Salida: crear fila (leads)"

Esto hace el escenario mantenible.

5.2 Diseña ya una ruta de revisión

Aunque el escenario sea simple, decide:

- qué pasa si falta un campo,
- qué pasa si hay un caso sensible,
- qué pasa si no se puede continuar.

Ejemplo: si falta email, envía aviso a un responsable en lugar de fallar silenciosamente.

5.3 Controla duplicados desde el inicio

Duplicados son el fallo nº1 en automatización por eventos.

Si puedes, guarda un "ID procesado" y rechaza repeticiones.

5.4 Mantén una "entrada de prueba" fija

Guarda un payload ejemplo y úsalo siempre. Así puedes comparar v1 vs v2.

6) Evidencias de que tu escenario está "bien"

Tu escenario está listo para crecer si:

- puedes ejecutarlo 3 veces con entradas similares y produce salidas coherentes,
- sabes dónde mirar si falla (historial, bundles),
- tienes control de datos mínimos,
- no duplica resultados.

Relación de ideas clave

- Make se organiza en escenarios (procesos) y módulos (pasos); los datos viajan en bundles.
- El trigger define el comportamiento: Scheduler (tiempo), Webhook (evento), Watch (vigilancia).
- Primer objetivo: escenario reproducible y verificable, no "demo bonita".
- Controla duplicados, valida datos y registra trazabilidad desde el inicio.

Más información en ChatGPT

- "Ayúdame a elegir el trigger correcto (Scheduler/Webhook/Watch) para este proceso: [describe proceso], con riesgos y mitigación."
- "Dame un checklist de pruebas para mi primer escenario en Make (entrada, bundles, validación, duplicados y salida visible)."

5.3.2 [Make] Datos y lógica del flujo: mapping/funciones + Router/Filters + listas (bundles, Iterator/Aggregator)

Referencia

- **Módulo:** 5
- **Apartado:** 5.3. Make: construcción de automatizaciones
- **Ficha:** 5.3.2 [Make] Datos y lógica del flujo: mapping/funciones + Router/Filters + listas (bundles, Iterator/Aggregator)

Palabras clave

Make mapping, funciones Make, Router Make, filtros Make, bundles Make, Iterator Make, Aggregator Make, normalización datos, rutas revisión, validación salida, eficiencia operaciones

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Entender y aplicar **mapping** (mapeo de campos) y **funciones** para transformar datos con precisión.
2. Diseñar lógica de negocio con **Router** (ramificación) y **Filters** (condiciones), incluyendo ruta principal y ruta de revisión.
3. Gestionar correctamente **listas**: reconocer múltiples bundles, usar **Iterator** para iterar y **Aggregator** para recombinar.
4. Evitar costes innecesarios y errores típicos: iteraciones masivas, filtros mal colocados, datos sin normalizar y salidas inconsistentes.

Síntesis

En Make, un escenario funciona bien cuando controlas tres cosas: **datos**, **decisiones** y **listas**. El mapping y las funciones permiten normalizar y crear un “contrato” de datos. El Router y los filtros permiten dirigir casos por prioridad o riesgo. Y los iteradores/agregadores permiten tratar listas sin romper el flujo ni disparar operaciones. Esta ficha te enseña a diseñar un flujo operable: principal + revisión + control de listas, con recomendaciones para eficiencia y trazabilidad.

Contenido principal

1) Mapping: qué es y por qué es el centro del flujo

Mapping es asignar campos de salida de un módulo a campos de entrada del siguiente. Es como rellenar un formulario automáticamente.

Ejemplo: el módulo “Watch Emails” entrega subject, from, body.

El módulo “Create Ticket” necesita title, requester, description.

El mapping decide qué va dónde.

Consejo: define un “contrato de datos” (ver 5.1.2) y mapea siempre hacia ese contrato. Si cada flujo usa nombres distintos, el mantenimiento se complica.

2) Funciones: pequeñas transformaciones que evitan grandes errores

Una **función** es una operación sobre un dato: recortar texto, concatenar, convertir fecha, comprobar si está vacío.

Ejemplos típicos:

- limpiar espacios,
- unir nombre + apellidos,
- convertir fecha a formato ISO,
- convertir “Sí/No” en true/false.

Recomendación operativa: transforma lo antes posible a un formato estándar. Si normalizas tarde, ya habrás tomado decisiones con datos inconsistentes.

3) Router y Filters: lógica de negocio con rutas claras

3.1 Router (ramificación)

Un **Router** divide el flujo en ramas. Es “si pasa X, ve por aquí; si pasa Y, por allí”.

Úsalo para separar:

- **ruta principal** (casos normales),
- **ruta revisión** (faltan datos, riesgo alto, incertidumbre),
- **ruta descarte** (spam, duplicado).

3.2 Filters (condiciones)

Un **Filter** es una condición para que un bundle pase por una ruta:

- priority = alta,
- pedido_id existe,
- sensitive_flag = true.

Consejo clave: coloca filtros pronto, antes de pasos caros (IA, HTTP). Filtrar tarde aumenta coste y ruido.

4) Diseñar una ruta de revisión (calidad) como parte del flujo

La revisión es el “humano en el bucle” (human-in-the-loop). No es fracaso: es control.

Triggers típicos para revisión:

- falta un campo mínimo,
- el texto es ambiguo,
- la IA devuelve salida incompleta,
- hay palabras sensibles (“denuncia”, “fraude”, “reembolso”).

Recomendación: define una tabla sencilla de reglas de revisión y aplica esas reglas con filtros. Esto hace el flujo explicable y auditable.

5) Bundles y listas: el punto donde se disparan operaciones (y costes)

En Make, si un módulo devuelve varios resultados, aparecen varios **bundles**. Cada bundle puede recorrer el resto del flujo.

Riesgo nº1: sin darte cuenta, procesas 200 bundles y ejecutas 200 acciones (200 tickets, 200 emails).

Consejo: limita resultados al inicio y usa filtros.

6) Iterator y Aggregator: iterar y recombinar (concepto + uso)

6.1 Iterator (iterar una lista)

Un **Iterator** toma una lista (array) y la convierte en bundles, uno por elemento.

Útil cuando tienes, por ejemplo, 10 adjuntos y necesitas procesarlos uno a uno.

Recomendaciones:

- limita el tamaño de la lista,
- si la lista es grande, procesa en lotes o deriva a revisión.

6.2 Aggregator (agregar resultados)

Un **Aggregator** hace lo contrario: toma varios bundles y los reúne en uno (por ejemplo, crear un resumen final o una tabla).

Caso típico: buscar 20 registros, procesarlos, y luego generar un único resumen para enviar por email.

Consejo: agrega cuando la salida debe ser una sola cosa (un informe, un resumen). Si necesitas 20 acciones independientes, no agregues.

7) Estrategias de eficiencia (para no “quemar” operaciones)

- Filtra al inicio (antes de IA y HTTP).
- Evita iterar listas enormes.
- Usa agregación cuando el resultado final es único.
- Divide en escenarios: uno para capturar/normalizar, otro para procesar pesado.
- Añade límites: “si hay más de 50 items → revisión”.

Esto es fundamental en Make porque las operaciones suelen tener coste por ejecución.

8) Validación de salida (especialmente cuando hay IA)

Si un paso de IA devuelve:

- categoria, prioridad, resumen, pedido_id, valida:
- que existen campos obligatorios,
- que prioridad sea una de las permitidas,
- que si falta algo, sea “no indicado” y active revisión.

Recomendación: nunca uses directamente una salida IA para ejecutar acciones críticas sin validación y confirmación.

Relación de ideas clave

- Mapping y funciones convierten datos “sucios” en un contrato estándar.
- Router/Filters diseñan rutas de negocio y revisión.
- Bundles son unidades de ejecución: si hay muchos, aumentan coste y riesgo.

Programa Superior en Inteligencia Artificial Aplicada a la Empresa

Fichas de contenido

- Iterator y Aggregator permiten tratar listas con control.
- Validación y eficiencia son parte del diseño, no un añadido.

Más información en ChatGPT

- “Diseña reglas de Router/Filters para este caso: [describe], incluyendo ruta principal, revisión y descarte, con condiciones claras.”
- “Propón un contrato de salida para un paso de IA en Make (campos obligatorios, valores permitidos) y cómo validarlo.”

5.3.3 [Make] Integraciones y operación mínima: HTTP, autenticación, errores, logging, alertas y lectura de ejecuciones

Referencia

- **Módulo:** 5
- **Apartado:** 5.3. Make: construcción de automatizaciones
- **Ficha:** 5.3.3 [Make] Integraciones y operación mínima: HTTP, autenticación, errores, logging, alertas y lectura de ejecuciones

Palabras clave

Make HTTP module, autenticación API, tokens, OAuth, API key, errores 401 403 429 5xx, reintentos, backoff, logging, alertas, historial ejecuciones Make, depuración escenarios

Objetivos de aprendizaje

Al finalizar la lectura, deberás ser capaz de:

1. Integrar un sistema “sin conector” usando **HTTP** (peticiones a APIs) de forma segura y reproducible.
2. Entender los métodos HTTP básicos (GET/POST) y la autenticación típica (**API key**, **Bearer token**, **OAuth** explicado).
3. Reconocer y tratar errores comunes (401/403/429/5xx), diseñando rutas de error y reintentos controlados.
4. Implementar operación mínima: logging/trazabilidad, alertas, y lectura del historial de ejecuciones para depurar.

Síntesis

Cuando no existe un conector, la integración universal se hace con HTTP: el escenario “habla” con una API. Esto exige disciplina: autenticar bien, manejar errores y crear trazabilidad. En Make, el éxito en producción depende de la operación mínima: rutas de error, registros, alertas y capacidad de depuración desde el historial de ejecuciones. Esta ficha te enseña el marco práctico para integrar y operar un escenario real sin caer en fallos silenciosos.

Contenido principal

1) HTTP y API: qué significa “integrar sin conector”

HTTP es el protocolo de comunicación de la web. Una **API** (*Application Programming Interface*) es una interfaz para que un sistema permita a otro:

- pedir datos,
- crear registros,
- actualizar estados.

En automatización, “integrar sin conector” significa: usar un módulo HTTP para hacer una petición a una API y trabajar con la respuesta (normalmente JSON).

Consejo: antes de construir, define qué necesitas:

- leer (GET),
- crear (POST),
- actualizar (PUT/PATCH),
- borrar (DELETE, menos frecuente en automatizaciones).

2) Métodos HTTP básicos (explicado con ejemplos de negocio)

- **GET:** “dame información” (consultar estado de pedido).
- **POST:** “crea algo” (crear ticket o lead).

En Make, estos métodos se configuran en el módulo HTTP.

Recomendación: prueba primero con GET (más seguro), y cuando tengas control de datos, pasa a POST (acciones).

3) Autenticación: cómo demostrar quién eres ante la API

Sin autenticación, la API no te deja acceder.

Tipos comunes:

3.1 API Key (clave de API)

Es una clave (como un “password” de aplicación). Se envía normalmente en una cabecera (header) o como parámetro.

Riesgo: si se filtra, alguien puede usarla.

Consejo: guárdala en credenciales/variables seguras, no en texto visible en módulos.

3.2 Bearer token

Es un token que se envía en la cabecera:

Authorization: Bearer <token>

Puede caducar.

Consejo: planifica renovación y manejo de expiración.

3.3 OAuth (explicación sencilla)

OAuth es un protocolo para autorizar acceso sin compartir tu contraseña. Se usa mucho con Google/Microsoft. Implica:

- conceder permisos,
- recibir tokens,
- renovar tokens.

Consejo práctico: si el conector oficial existe, suele simplificar OAuth. Si vas por HTTP, prepara pruebas y control de expiración.

4) Manejo de errores: los códigos que debes reconocer

Los códigos HTTP indican qué pasó:

- **401 (Unauthorized):** credenciales incorrectas o token caducado.
Acción: revisar credenciales, refrescar token.

- **403 (Forbidden):** tienes credenciales, pero no permisos.
Acción: revisar roles/permisos en el sistema.
- **429 (Too Many Requests):** demasiadas peticiones en poco tiempo (rate limit).
Acción: aplicar reintentos con espera (*backoff*), reducir frecuencia, usar batch.
- **5xx:** error del servidor (servicio caído o inestable).
Acción: reintentar con límite y alertar si persiste.

Recomendación operativa: no trates todos los errores igual. 401/403 se resuelven con credenciales/permisos; 429/5xx con reintentos y control de carga.

5) Reintentos y backoff (esperas inteligentes)

Un **reintento** es volver a intentar una petición.

Backoff es esperar un tiempo antes de reintentar (por ejemplo, 5s, luego 15s, luego 60s).

Por qué importa: sin backoff, puedes empeorar el problema (más 429) o generar costes.

Regla recomendada: pocos reintentos (2–3) y luego ruta de error con alerta.

6) Diseño de rutas de error (para que falle con control)

Tu escenario debe tener:

- ruta principal,
- ruta revisión (negocio),
- ruta error técnico (HTTP/API).

En error técnico:

- registra `trace_id`,
- guarda el código y el mensaje,
- notifica (email/Slack/Teams),
- y si procede, crea un ticket interno.

Consejo: evita que el flujo “muera” sin avisar. El error silencioso es el peor.

7) Logging y alertas: operación mínima indispensable

Logging es registrar información para depurar. En Make, además de logs implícitos del historial, conviene que tu salida o tus registros incluyan:

- `trace_id`,
- `source`,
- resultado (OK/revisión/error),
- timestamp.

Alertas: notificaciones cuando:

- hay error técnico,
- hay demasiados casos en revisión,
- hay picos de volumen,
- o se detectan datos sensibles.

Recomendación: empieza con alertas simples (email a responsable). Luego evoluciona a un canal operativo (Teams/Slack).

8) Lectura de ejecuciones en Make: cómo depurar sin adivinar

El historial de ejecuciones te permite ver:

- en qué módulo falló,
- qué bundle entró,
- qué devolvió la API,
- y qué datos se mapearon.

Método de depuración recomendado:

1. identifica trace_id o ejecución concreta,
2. abre el módulo fallido y revisa entrada/salida,
3. compara con tu contrato de datos (campos obligatorios),
4. corrige: mapping, validación, reintentos o credenciales,
5. reejecuta con la misma entrada de prueba.

Consejo: conserva un payload de prueba (webhook) o un caso ejemplo. Depurar sin caso reproducible es perder tiempo.

9) Checklist final: integración HTTP “lista para producción”

- ¿Método correcto (GET/POST) y endpoint validado?
- ¿Autenticación segura (no hardcode)?
- ¿Validación de respuesta (JSON esperado)?
- ¿Manejo de 401/403/429/5xx con rutas y reintentos?
- ¿Logs con trace_id y alertas?
- ¿Ejecución reproducible con caso de prueba?

Relación de ideas clave

- HTTP + API permite integrar cualquier sistema, pero exige disciplina.
- Autenticación típica: API key, Bearer token u OAuth (permisos y tokens).
- Códigos 401/403/429/5xx se tratan de forma distinta.
- Reintentos con backoff + ruta de error evitan fallos silenciosos.
- Logging, alertas y lectura de ejecuciones son la operación mínima real.

Más información en ChatGPT

- “Diseña una estrategia de manejo de errores para un módulo HTTP en Make: qué hacer con 401/403/429/5xx, reintentos, backoff, alertas y logs.”
- “Ayúdame a definir un contrato de entrada/salida para una llamada HTTP a una API de tickets, con validación y ruta de error.”